CampusResolve - AI Prompt History

Disclosure and scope

This document contains reconstructed and consolidated prompts describing the AI-assisted development workflow. It is **not a verbatim export** of the complete ChatGPT conversation. The original conversation includes additional debugging questions, code exchanges, and follow-up instructions. These 21 entries summarize the major project stages and should not be represented as every prompt submitted to AI tools.

YottaFlex requests the actual prompts submitted to AI tools. Where available, the original prompt messages or conversation export should be provided separately to satisfy that requirement. AI responses are not included in this document.

Prompt 01

Act as an experienced software engineer with good knowledge of Python, Flask, HTML, CSS, JavaScript, SQL, and AI applications. I have received a coding challenge from YottaFlex AI, and I want to build CampusResolve, a Student and Faculty Support Management System. Please help me understand the requirements and develop the project from scratch, step by step. I am still learning, so explain each step in simple English, give complete code for every file, and wait until I finish and test one step before moving to the next.

Prompt 02

I have shared the YottaFlex project requirements. Please read them carefully and explain what the project is expected to do. Tell me the important features, what files we need, what data we need to generate, and how we can complete everything using free tools on my computer.

Prompt 03

I want to use Python Flask for the backend, SQLite for the database, and HTML, CSS, and JavaScript for the frontend. Please suggest a simple folder structure and tell me exactly how to set up the project in VS Code on Windows. Give me one command at a time and explain what result I should see.

Prompt 04

Please help me create the database for CampusResolve. We need students and faculty, roles, academic terms, categories, tickets, messages, policies, knowledge articles, assignments, SLA details, resolution feedback, and ticket history. Make sure the tables are connected correctly using primary keys and foreign keys. Give me the full database.py file.

Prompt 05

Now help me build a Flask website where a student or faculty member can create a support ticket. The form should collect the necessary details and save the ticket in SQLite. After submitting, I want to see a unique ticket number and the ticket details. Please give me the complete code for any files that need to change.

Prompt 06

I want my CampusResolve website to look clean and professional. Use a calm green theme with the colors #243B35, #6B8E7B, and #F8FAF7. Create a dashboard, navigation, a ticket form, an All Tickets page, and a ticket details page. Keep the design consistent and easy to use.

Prompt 07

Please add automatic ticket routing. When someone creates a ticket under Financial Aid, Registration, IT, or Accessibility, the system should assign it to the correct support team. Check the routing rules before assigning a team, record the assignment in the ticket history, and help me test whether it works.

Prompt 08

I want CampusResolve to answer requests using stored policy documents instead of giving unsupported answers. Please create sample policies with version numbers and effective dates, find the appropriate policy for each request, and show the policy reference with the answer. If a suitable policy is not available, the ticket should be sent for human review.

Prompt 09

Please build an SLA feature for the support tickets. Our working hours are Monday to Friday, 9 AM to 5 PM IST. Calculate response and resolution deadlines using only business hours, including cases where a ticket is created after office hours or during the weekend. Please add tests for the calculations.

Prompt 10

Now add SLA monitoring and human escalation. If a ticket passes its deadline or cannot receive a supported policy answer, update its status appropriately and record what happened in the activity history. Please make sure the feature does not break the existing ticket workflow.

Prompt 11

The challenge requires a synthetic dataset with exactly 5,000 tickets, 200 knowledge articles, and 20,000 messages across four categories. Please write a Python generator using the fixed seed 20261022. Make the records realistic and connected correctly, with valid policy references, assignments, SLA records, and ticket outcomes. Keep this dataset separate from my live website tickets.

Prompt 12

Please create five intentional negative test cases in a separate copy of the synthetic database: an expired policy citation, an unauthorized team assignment, an unsupported answer, an SLA deadline that ignores business hours, and a missed escalation. Store the correct labels in a separate ground_truth.csv file so the original dataset stays unchanged.

Prompt 13

I want to verify that the generated data is correct. Please write validation code to check the number of records, duplicate IDs, foreign keys, valid ticket statuses, dates, routing permissions, policy validity, SLA calculations, and ticket lifecycle consistency. Show clear pass or fail results so I can understand the output.

Prompt 14

Please check whether the dataset is reproducible. Generate it twice in separate temporary folders using the same seed and compare the record counts and checksums. Give me a complete Python test file and tell me how to run it from the terminal.

Prompt 15

Please create a script to export the synthetic dataset into CSV, JSON, and SQL files. Include a manifest with record counts and checksums so a reviewer can verify the exported data. Make sure the exported tables and files are clearly named.

Prompt 16

I have 5,000 tickets in the synthetic database, but I also want to see them on the website. Please update the dashboard and All Tickets page to show the synthetic data. Add search, status filters, and pagination, while keeping live tickets separate. When I click a ticket number, it should open the correct ticket details.

Prompt 17

Please improve the ticket details page. I want to see the requester information, issue description, assigned team, ticket status, activity history, messages, policy citation with version and effective dates, and SLA deadlines. Keep the same green design and provide the full updated HTML file.

Prompt 18

Please create one command that runs all our existing tests together, including routing, policy answers, SLA, monitoring, dataset validation, negative cases, lifecycle checks, and reproducibility. I want to see which test groups passed and failed without running every file separately.

Prompt 19

We have finished the main CampusResolve features and the tests are passing. Please help me prepare a clear README and data dictionary for GitHub. Explain how to install the project, run Flask, generate the dataset, export files, run tests, and understand the limitations. Do not claim anything that we have not actually implemented or tested.

Prompt 20

YottaFlex says we must disclose how we used AI. Please help me write a simple and truthful AI disclosure explaining where ChatGPT helped with planning, coding, debugging, testing, and documentation, and what I personally ran and verified. I also need to include my actual AI prompt history, without the AI responses.

Prompt 21

Please help me prepare the CampusResolve project for a public GitHub repository. Create a safe .gitignore, check that the required source code, documentation, data generation scripts, validation reports, and prompt history are included, and guide me through uploading and submitting the repository link step by step.